SEEING LIKE A MACHINE

Table of Contents

SEEING LIKE A MACHINE

A Polytechnic Student's Journey Through Python, OpenCV, and Machine Learning

A Companion Textbook for DEC50302 — Machine Learning in Computer Vision

Prepared for the Department of Electrical Engineering, Politeknik Sultan Abdul Halim Mu'adzam Shah (POLIMAS)

How to Read This Book

Every textbook makes a promise to its reader. This one promises that you will never be asked to memorise a definition without first seeing why it matters. Machine learning in computer vision is, at its heart, a very human story: it is about teaching a machine to do something every sighted person does without thinking — to look at a scene and understand it. Diploma engineers are not asked to invent this field; you are asked to *use* it, confidently and correctly, to build things that work.

To keep that promise, this book follows one recurring character through every chapter: **Amir**, a Semester 5 Diploma student at a Malaysian polytechnic who has been given a real assignment by his Head of Department — build a working prototype of a "Smart Guardhouse" system for the campus entrance. The guardhouse must eventually recognise number plates, greet returning staff by face, follow a moving object with a camera, and flag visitors who forget their safety vest. It sounds ambitious for a diploma project. It is. But it is also *exactly* the kind of project this course is designed to produce, one small working piece at a time.

You will meet Amir at the start of every unit. His confusions are deliberate — they are the same confusions almost every student has at that stage — and his solutions are the same techniques you will practise in your own labs. Where Amir hits a real bug, that bug is shown on purpose, because debugging is where computer vision is actually learned.

This book is organised into three learning units that match your syllabus (**Python Foundations**, **OpenCV**, and **Machine Learning Applications**), followed by a projects unit that assembles everything into complete, working applications, an active-learning and assessment unit, and a short unit on the ethics you must carry with you once you can make a camera "see."

Three kinds of boxes appear throughout the text:

> 🔧 **Micro-Exercise** — A short, five-to-fifteen-minute task you should type and run yourself, right now, before reading on. Typing code is not optional in this field; your fingers learn things your eyes do not.
>
> 📋 **Lab Blueprint** — A structured, step-by-step build guide for a complete mini-application, written the way an engineer's build sheet is written: stage by stage, with the exact function to call at each stage.

> 🎯 **Assessment Link** — A note connecting the section you just read to the specific Continuous Assessment component (Practical Work, Mini Project, or Self-Learning Task) it prepares you for.

Your Recommended Companion Reference

Alongside this book, keep a copy of ***Building Computer Vision Applications Using Artificial Neural Networks*** by **Shamshad Ansari** (2nd Edition, Apress) within reach. Ansari's book is the deeper, industry-grade reference this course draws its philosophy from — it is written by a practising computer vision engineer for people who intend to *build* systems, not merely study them, which is precisely the spirit of this diploma. Wherever this textbook introduces a concept, look for the **"Go Deeper with Ansari"** note that tells you exactly which chapter to open if you want the fuller, more mathematical treatment — from image arithmetic in his Chapter 3, to convolutional neural networks in Chapter 5, to full object-detection pipelines with YOLO and TensorFlow in Chapters 6 through 9.

Preface: Why a Diploma Engineer Needs to Understand Sight

Somewhere on a factory floor in Kulim, a camera is watching a conveyor belt, counting defective solder joints faster than any human inspector could. Somewhere in a Klang Valley condominium, another camera reads a resident's face and unlocks a gate without anyone touching a card reader. Somewhere on a drone above a padi field, a third camera watches for irrigation leaks. None of these cameras are "smart" on their own — a camera is just a sensor. What makes each of these systems intelligent is a layer of *machine learning* sitting on top of the pixels, translating raw light into decisions.

This textbook, designed specifically for Diploma students under the Department of Polytechnic and Community College Education, is the definitive resource for **DEC50302 Machine Learning in Computer Vision**. The course is anchored by three Course Learning Outcomes: you must be able to **apply** machine learning methods using Python and OpenCV; you must be able to **construct** working applications from those methods; and you must demonstrate the ability to **keep learning independently**, because the tools in this field change every year and the engineer who stops reading documentation stops being useful.

The curriculum bridges two very different ways of thinking. In your earlier programming courses, you told the computer exactly what to do, step by step. In this course, for the first time, you will start *showing* the computer examples and letting it discover the rule for itself. That shift — from explicit instruction to data-driven learning — is the single biggest idea in this book, and every unit circles back to it.

Structurally, the book follows the syllabus exactly: **Unit 1** builds the Python foundation you need, framed specifically around the way pixels and image data behave. **Unit 2** teaches OpenCV, the toolbox that lets Python actually touch and manipulate images and video. **Unit 3** introduces the machine learning layer that turns processed pixels into decisions. **Unit 4** puts all three together into four complete, buildable projects. **Unit 5** gives you active-learning activities and shows you exactly how your Continuous Assessment (100%, split across Practical Work, a Mini Project, and Self-Learning Tasks) maps onto everything you've studied. **Unit 6** closes with the ethical responsibilities that come with the power to make a machine see.

By the last page, you should be able to look at a blinking cursor and a webcam and know exactly what to build.

UNIT 1 — Foundations of Python Programming: Learning to Speak the Machine's Language

Amir's Notebook, Week 1

Amir's lecturer had written a single sentence on the whiteboard: *"Before your guardhouse can see anything, it has to be able to think."* Amir didn't understand what that meant until his first lab, when he tried to write a script that stored a camera's brightness reading and Python cheerfully let him store a word instead. No error. No warning. It just worked — which was somehow more alarming than if it had crashed.

That was Amir's first real lesson about Python: it trusts you completely, and trust without discipline is how bugs are born.

> 📎 **Practical Work Link** — This unit pairs with **Practical Work 1 (Introduction to Python Programming)** and the first half of **Practical Work 2 (Python Basics for Image Processing)** in *Amir's Lab Companion*, the step-by-step lab volume that accompanies this textbook.

1.1 The Python Paradigm

1.1.1 Why an Interpreted Language Wins for Computer Vision

Python is an interpreted, high-level, general-purpose language. Unlike compiled languages such as C or Java, which translate an entire program into machine code before anything runs, Python executes code

line by line through an interpreter. For a compiled language, changing one number means recompiling the whole program before you see the result. For computer vision development, where you are constantly nudging a threshold value by five and immediately wanting to see the new output on screen, this difference is not a technicality — it is the reason rapid prototyping is even possible. A developer can change a blur radius, save the file, and watch the new result in under a second.

Python's design philosophy also emphasises readability through significant indentation. There are no curly braces marking the start and end of a block; the visual shape of the code *is* its logical shape. This matters more in computer vision than it sounds, because the algorithms you will write soon involve nested loops and conditionals over image regions, and a misplaced indent is a bug that will not throw an error — it will simply produce a wrong answer.

1.1.2 Setting Up the Environment

Every polytechnic lab begins the same way: an empty folder and a decision about tools. For this course, we standardise on **Visual Studio Code (VS Code)**, because its extension ecosystem lets you write Python, manage Jupyter notebooks, and even remote into a cloud GPU for training — all from one place. Step-by-step installation instructions are maintained at https://code.visualstudio.com/docs/python/python-tutorial.

The setup has three parts:

1. **Install the Python interpreter.** Always install the latest stable Python 3 release.
2. **Install packages with pip.** The Python Package Installer fetches ready-made libraries from the Python Package Index (PyPI). For this course, your first command will almost always be:

```bash
pip install opencv-python numpy tensorflow
```

3. **Use a virtual environment.** Best practice is to isolate every project's dependencies using `venv` or `conda`, so that installing a new library for one project cannot silently break another.

> After installation, sign in to Microsoft's free introductory Python course at https://vscodeedu.com/courses/intro-to-python using your polytechnic email (`id@polimas.edu.my`). POLIMAS students receive free access under Microsoft Education, and the course requires nothing more than a browser and an internet connection.

> **Go Deeper with Ansari** — Ansari's Chapter 1, *Prerequisites and Software Installation*, walks through the equivalent setup for PyCharm, virtualenv, and OpenCV 4 on Windows, macOS, and Linux, and is a useful second opinion if your lab machine behaves differently than expected.

1.2 Common Programming Elements — Seen Through a Pixel's Eyes

1.2.1 Variables and Dynamic Typing

In Python, a variable is a label pointing at an object in memory, not a fixed box with a declared type. The same name can hold an integer at one moment and a string the next. That flexibility is exactly what

tripped Amir up — and it is exactly the behaviour you must learn to control deliberately rather than suffer accidentally.

Core data types you will use constantly:

- **Integers (`int`)** — whole numbers for counting iterations, pixel coordinates, or class labels (`class_id = 1`).
- **Floating-point numbers (`float`)** — decimal precision, ubiquitous in computer vision for confidence scores (`0.98`), scaling factors, and normalised pixel values.
- **Strings (`str`)** — sequences of characters for file paths (`"data/image.jpg"`), labels, and interface text.
- **Booleans (`bool`)** — `True`/`False` values essential for control flow, such as checking whether a camera frame was successfully captured (`ret, frame = cap.read()`).

> 🔧 **Micro-Exercise 1.1 — The Lifecycle of a Pixel**
>
> *Objective: watch a single value change type as it moves through a mini image-processing pipeline.*

Python

```python
# Micro-Exercise 1.1: The Lifecycle of a Pixel
# Objective: demonstrate dynamic typing by simulating
pixel normalisation.

# 1. Integer: a raw grayscale pixel value (0-255)
pixel_val = 145
print(f"Stage 1 - Raw Input: {pixel_val}, Type:
{type(pixel_val)}")
# Expected: <class 'int'>

# 2. Float: normalisation, crucial before feeding a
neural network
pixel_val = pixel_val / 255.0
print(f"Stage 2 - Normalized: {pixel_val:.4f}, Type:
{type(pixel_val)}")
# Expected: <class 'float'>

# 3. List: a row of pixels (a "scanline")
pixel_row = [12, 240, 88, 3]
print(f"Stage 3 - Pixel Row: {pixel_row}, Type:
{type(pixel_row)}")
```

> This exercise isolates dynamic typing from the complexity of the OpenCV library, so you can watch type coercion (int → float) happen in a controlled setting before meeting it inside a much larger, harder-to-debug NumPy array.

1.2.2 Operators and Precedence

Python follows standard mathematical precedence (PEMDAS) — a rule that matters enormously once you start writing equations for image transformations or loss functions.

Operator Type	Symbol	Precedence	Description
Exponentiation	`**`	Highest	Raises a number to a power — used when calculating squared errors in ML
Multiplicative	`*`, `/`, `//`, `%`	High	Multiplication, division, floor division, modulus — used for resizing and aspect-ratio math
Additive	`+`, `-`	Medium	Addition and subtraction — used for brightness adjustment and matrix operations
Relational	`<`, `>`, `<=`, `>=`	Low	Comparisons — used for thresholding, e.g. `pixel_value > 127`
Logical	`not`, `and`, `or`	Lowest	Boolean logic — used to combine detection conditions

Active Learning Insight: In `5 + 2 * 3`, Python evaluates the multiplication first (`2 * 3 = 6`), then the addition (`5 + 6 = 11`). If you write `x = width / 2 + 10`, the division happens *before* the addition — so to calculate the midpoint of two coordinates, parentheses are not optional: `midpoint = (a + b) / 2`.

1.2.3 Control Flow: Conditionals and Loops as the Grammar of Vision

Control flow statements are how logic actually gets implemented.

- **Conditionals (`if`, `elif`, `else`)** execute a block based on a truth value. A face-detection routine, for instance, checks `if len(faces) > 0:` before attempting to draw a bounding box.
- **`for` loops** iterate over a known sequence — perfect for stepping through a list of detected objects or the pixels inside a region of interest.

- **`while` loops** repeat as long as a condition holds. The pattern `while True:` is the standard architecture for a live video feed: the loop keeps capturing frames until the user presses a key to stop it.

Understanding loops is a direct prerequisite to understanding **convolution**, the mathematical operation at the heart of every image filter you will use in Unit 2. Convolution is, underneath all the OpenCV convenience functions, just a nested loop sliding a small window across every pixel.

> 🔧 **Micro-Exercise 1.2 — Thresholding Logic Simulation**
>
> *Objective: use a `for` loop to apply binary threshold logic to a simulated pixel row, without OpenCV.*

Python

```python
# Micro-Exercise 1.2: Thresholding Logic Simulation

image_row = [20, 150, 45, 200, 10, 127, 180, 90]   # simulated grayscale scanline
threshold_value = 127
binary_row = []

print("Original Scanline:", image_row)

for pixel in image_row:
    # If the pixel is bright (> 127), make it White (255); otherwise Black (0)
    if pixel > threshold_value:
        binary_row.append(255)
    else:
        binary_row.append(0)

print("Thresholded Scanline:", binary_row)
```

> This connects the abstract idea of a loop to the concrete computer vision task of **thresholding**. When you later call `cv2.threshold()` in Unit 2, you will already understand the algorithmic logic running underneath it.

1.2.4 Data Structures: Choosing the Right Container

Python offers four built-in structures that differ in ordering and mutability, and choosing the wrong one is one of the most common causes of slow or buggy computer vision code.

Structure	Syntax	Ordered?	Mutable?	Typical Use in Computer Vision
List	`[1, 2, 3]`	Yes	Yes	Storing bounding boxes `[[x,y,w,h],`

Structure	Syntax	Ordered?	Mutable?	Typical Use in Computer Vision
				`...]`, sequences of frames, or contour points
Tuple	`(1, 2, 3)`	Yes	No	Storing fixed constants such as image dimensions `(width, height)` or a colour `(255, 0, 0)`
Dictionary	`{'key': val}`	Yes (3.7+)	Yes	Mapping class IDs to labels, e.g. `{'0': 'cat', '1': 'dog'}`, or storing configuration parameters
Set	`{1, 2, 3}`	No	Yes	Storing unique object IDs currently in frame, to prevent double-counting

A picture to hold in your head: a *list* is a train with numbered carriages — passengers (data) can change seats. A *tuple* is a sealed shipping container — once packed, the contents are fixed. A *dictionary* is a library where you find a book not by its shelf position, but by looking up its title (the key).

> 🔧 **Micro-Exercise 1.3 — Coordinates and Labels**
>
> *Objective: build, slice, and modify a list of detected-object coordinates.*

Python

```
# A list of detected object coordinates: [x, y, label]
detections = [[120, 45, "car"], [300, 200, "person"],
[10, 400, "plate"]]

# Slicing: get everything except the last detection
print("All but last:", detections[:-1])
```

Python

```
# Modifying in place (lists are mutable)
detections[0][2] = "vehicle"       # relabel the first
detection
print("After relabel:", detections)

# A tuple for a fixed frame size — cannot be modified
once created
frame_size = (640, 480)
# frame_size[0] = 1280   # <- this line would raise a
TypeError
```

1.3 Functions and Modules: Building Reusable Vision Tools

A function packages a block of logic under one name so that logic can be reused rather than retyped. Modules go one step further, grouping related functions into an importable file (`import cv2` is exactly this — it pulls in the entire OpenCV toolbox). As your guardhouse project grows past a single script, functions and modules are what keep the code from collapsing into an unreadable wall of copy-pasted logic.

1.4 Object-Oriented Programming: The Detector Paradigm

Diploma syllabuses often teach Object-Oriented Programming (OOP) with generic examples — a `Dog` class that inherits from an `Animal` class. Those examples are fine for learning syntax, but they do not transfer to project work. In this course, OOP is taught through the **Detector Paradigm**: every vision algorithm you build is, underneath, an object that can be *instantiated*, that *encapsulates* its own settings, and that can be *specialised* through inheritance.

1.4.1 Encapsulation and Instantiation

Python

```
class FaceDetector:
    def __init__(self, model_path):
        self.model_path = model_path      # attribute

    def detect(self, image):
        # logic to detect faces goes here
        pass

# Instantiation
my_detector = FaceDetector("haarcascade.xml")
```

`__init__` is the constructor — it runs automatically the moment a new object is created, and it is where an object's attributes are initialised.

Textbook Blueprint — the `ShapeDetector` class, which you will use directly in Unit 4:

Python

```python
import cv2

class ShapeDetector:
    def __init__(self):
        # Encapsulation: the detection logic is hidden
inside the class
        pass

    def detect(self, contour):
        """Approximates a contour to identify its
shape."""
        shape = "unidentified"

        # 1. Calculate the perimeter
        peri = cv2.arcLength(contour, True)

        # 2. Approximate the contour to a simpler
polygon.
        #    0.04 is a hyperparameter encapsulated inside
the method.
        approx = cv2.approxPolyDP(contour, 0.04 * peri,
True)

        # 3. Classify the shape by counting vertices
        if len(approx) == 3:
            shape = "triangle"
        elif len(approx) == 4:
            (x, y, w, h) = cv2.boundingRect(approx)
            aspect_ratio = w / float(h)
            shape = "square" if 0.95 <= aspect_ratio <=
1.05 else "rectangle"
        else:
            shape = "circle"

        return shape
```

The `ShapeDetector` class demonstrates encapsulation by hiding the complexity of the Ramer–Douglas–Peucker approximation algorithm (`cv2.approxPolyDP`) behind one clean method call, `.detect()`.

1.4.2 Access Levels: Public, Protected, Private

Python signals access intent through naming convention rather than strict enforcement:

- **Public** (`self.name`) — accessible from anywhere.
- **Protected** (`self._name`) — intended for internal use by the class and its subclasses.
- **Private** (`self.__name`) — inaccessible from outside the class (enforced through name mangling). This matters for hiding model weights or configuration values that should never be manually altered by a user of the class.

1.4.3 Inheritance and Polymorphism: The Detector Hierarchy

The syllabus requires you to *instantiate, encapsulate,* and *inherit* — the cleanest way to demonstrate all three at once is a generic `BaseDetector` superclass that handles the shared plumbing, with specific detector subclasses beneath it.

Python

```
class BaseDetector:
    """Superclass: defines the interface every detector
must follow."""
    def __init__(self, algorithm_name):
        self.algorithm = algorithm_name
        print(f"Loading {self.algorithm} Detector...")

    def process(self, image):
        # Polymorphism: subclasses override this method
        raise NotImplementedError("Subclasses must
implement process()")


class FaceDetector(BaseDetector):
    """Subclass specialised for Haar Cascade face
detection."""
    def __init__(self, cascade_path):
        super().__init__("Haar Cascade Face")
# call the superclass constructor
        self.classifier =
cv2.CascadeClassifier(cascade_path)

    def process(self, image):
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        return self.classifier.detectMultiScale(gray,
1.1, 4)


# Usage demonstrates polymorphism: every detector
responds to .process()
```

```python
# the same way, regardless of its internal
implementation.
detectors =
[FaceDetector("haarcascade_frontalface_default.xml")]
for d in detectors:
    d.process(frame)
```

This structure mirrors the design pattern used inside major libraries such as PyTorch and Keras, which is exactly why it is worth learning now — it prepares you for Unit 3.

1.4.4 Magic Methods: Letting Objects Describe Themselves

Magic (or "dunder") methods let an object interact naturally with Python's built-in operators: `__str__` defines how it prints, `__len__` defines what `len(object)` returns, and `__add__` defines what + does to it.

> 🔧 **Micro-Exercise 1.4 — A Self-Describing Result**

```python
class Result:
    def __init__(self, label, conf):
        self.label = label
        self.conf = conf

    def __str__(self):
        # Called automatically whenever print(object)
runs
        return f"Detected: {self.label} (Confidence:
{self.conf*100:.1f}%)"

r = Result("vehicle", 0.93)
print(r)   # Detected: vehicle (Confidence: 93.0%)
```

> This one method turns a debugging session from a wall of memory addresses into a readable sentence — a habit every professional Python developer keeps for life.

> 🎯 **Assessment Link — Unit 1**
>
> This unit prepares you for: - **Practical Work (40%):** writing modular, error-free Python scripts that use loops and conditionals to process image data. - **Mini Project (Knowledge, 25%):** justifying your OOP design choices — e.g., "why did you build a separate class for your detector?" - **Self-Learning Task (CLO3):** search PyPI for the `imutils` library, install it, and write a script comparing `imutils.resize()` with `cv2.resize()`. Summarise, in your own words, how the library abstracts away the aspect-ratio math.

UNIT 2 — Introduction to OpenCV: Giving the Machine Its Eyes

Amir's Notebook, Week 3

The first time Amir ran `cv2.imread("gate.jpg")` on a file that didn't exist, nothing happened. No crash, no error — just a blank grey window and a program that refused to do anything useful. It took him forty minutes of confused staring before his lab partner pointed out the obvious: `imread` doesn't throw an exception when a file is missing. It just quietly returns `None`, and everything downstream fails in strange, hard-to-trace ways.

"OpenCV doesn't shout when something goes wrong," his lecturer told him afterward. *"It whispers. You have to learn to listen for the silence."* That single lesson — defensive coding, checking your assumptions before trusting them — turns out to matter more than any single OpenCV function you will learn this semester.

OpenCV (Open Source Computer Vision Library) is the industry-standard toolkit for real-time computer vision. It gives Python the machinery to load, manipulate, filter, and analyse visual data — everything from simple image filtering to full geometric transformation.

> 📎 **Practical Work Link** — This unit pairs with the second half of **Practical Work 2** and all of **Practical Work 3 (OpenCV Image and Video Processing)** in *Amir's Lab Companion*.

2.1 Tutorial 2.1: Constructing Code for Still Images and Video

Step 1 — Robust Image Loading

Python

```
import cv2
import sys

# 1. Load Image
img_path = "dataset/sample.jpg"
img = cv2.imread(img_path)

# 2. Validation — this is the habit Amir learned the hard
way
if img is None:
    sys.exit(f"Error: Could not load image at {img_path}.
Check the file path.")

# 3. Display
cv2.imshow("Display Window", img)
```

```python
cv2.waitKey(0)          # waits indefinitely for a key press
cv2.destroyAllWindows()
```

Step 2 — The Video Loop Architecture. This five-step skeleton is the backbone of *every* real-time application you will build for the rest of this course. Memorise its shape, not just its syntax.

```python
# 1. Initialise capture (index 0 is usually the default webcam)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    sys.exit("Error: Webcam not accessible.")

while True:
    # 2. Frame capture
    ret, frame = cap.read()
    if not ret:
        print("Stream ended.")
        break

    # 3. Processing — your algorithms go here
    # e.g., gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # 4. Visualisation
    cv2.imshow("Live Feed", frame)

    # 5. Exit logic — wait 1 ms for the 'q' key
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Table 2.1 — Common `VideoCapture` Troubleshooting

Symptom	Probable Cause	Fix
`AttributeError: 'NoneType' object has no attribute ...`	Camera index is wrong, or the camera is already in use by another program	Try `cv2.VideoCapture(1)`, or check OS privacy/camera permissions

Symptom	Probable Cause	Fix
Window freezes / stops responding	Missing `cv2.waitKey(1)` inside the loop	Ensure `waitKey(1)` sits inside the `while` loop, not outside it
"Assertion failed" error	Image path is wrong or the file is corrupted	Use `os.path.exists(img_path)` to verify the path before loading

> **Go Deeper with Ansari** — Chapter 2, *Core Concepts of Image and Video Processing*, covers pixel basics and coordinate systems with additional worked examples for loading, exploring, and drawing on images.

2.2 Tutorial 2.2: Pixel Representation and Colour Spaces

The coordinate system. OpenCV's origin `(0, 0)` sits at the **top-left** corner of the image. The x-axis increases rightward; the y-axis increases *downward*. This is the opposite of the Cartesian graphs you learned in mathematics class, where y increases upward — and forgetting this single fact is a leading cause of "my bounding box is upside-down" bugs.

Colour spaces are mathematical models for representing colour:

- **BGR** — OpenCV loads images in Blue-Green-Red order by default (not RGB). This additive colour model is the default for every `imread()` call.
- **Grayscale** — represents brightness only. Converting to grayscale (`cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`) is a standard first step because it collapses three colour channels into one, dramatically speeding up tasks like edge detection where colour information is irrelevant.
- **HSV (Hue, Saturation, Value)** — a cylindrical model that aligns much more closely with human colour perception, and is far better than BGR for colour-based object tracking.
  - **Hue (0–179)** — the pigment or colour identity (red, green, blue).
  - **Saturation (0–255)** — the vibrancy or purity of the colour.
  - **Value (0–255)** — the brightness of the light.
  - **Why HSV wins:** a shadow falling across a red object shifts its R, G, *and* B values significantly in BGR. In HSV, a shadow mostly affects only the *Value* channel — the *Hue* (the colour identity) stays largely constant. This makes colour tracking far more resilient to changing light.

> 🔧 **Micro-Exercise 2.1 — The Blue-Channel Filter**
>
> *Objective: use NumPy slicing to isolate a single colour channel.*
>
> ‴python import cv2
>
> img = cv2.imread("sample.jpg")

OpenCV loads channels as BGR: index 0 = Blue, 1 = Green, 2 = Red

Accessing a pixel is img[y, x]

```
img[:, :, 1] = 0 # zero out Green img[:, :, 2] = 0 # zero out Red
cv2.imshow("Blue Channel Only", img) cv2.waitKey(0) ```
```

2.3 Image Processing Techniques

2.3.1 Masking

Masking isolates a Region of Interest (ROI). A mask is a binary image where white pixels (255) mark the area to keep and black pixels (0) mark the area to discard. `cv2.bitwise_and()` combines the original image with the mask to extract exactly that ROI.

2.3.2 Blurring (Smoothing)

Blurring suppresses high-frequency noise before further processing:

- **Gaussian Blur** — uses a bell-curve kernel; effective against general Gaussian noise.
- **Median Blur** — replaces each pixel with the median value of its neighbourhood; excellent for removing "salt-and-pepper" noise (random black/white specks) while preserving edges.
- **Bilateral Filter** — blurs while *preserving* sharp edges, by weighting both spatial proximity and intensity similarity. Prevents the "smudging" of object boundaries that plain blurring causes.

2.3.3 Canny Edge Detection

The Canny algorithm is the gold standard for edge detection, and it is a four-stage pipeline worth understanding stage by stage:

1. **Gaussian Blur** — reduce noise first, or the edge map will be full of false positives.
2. **Gradient Calculation** — determine the direction and rate of intensity change at every pixel.
3. **Non-Maximum Suppression** — thin the detected edges down to a single-pixel width by suppressing anything that isn't a local peak.
4. **Hysteresis Thresholding** — apply two thresholds. Strong edges (above the high threshold) are kept automatically; weak edges (between the two thresholds) are kept *only* if they connect to a strong edge. This continuity check is what keeps object boundaries intact instead of broken into dashes.

2.4 Tutorial 2.4: Writing Contour Code

Contours are curves joining continuous points of equal colour or intensity along a boundary — in practice, the outline of a shape in a binary image.

Pipeline:

1. **Binary threshold** — contours are found most reliably on black-and-white images.
2. **Find contours** — `cv2.findContours()` walks the topology of the binary image.
3. **Approximate** — `cv2.approxPolyDP()` simplifies each contour's shape.

Python

```
# 1. Preprocess
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 127, 255,
cv2.THRESH_BINARY)

# 2. Find contours
# RETR_EXTERNAL: only the outermost contours (ignores
holes)
# CHAIN_APPROX_SIMPLE: compresses straight line segments
to save memory
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

# 3. Approximate and classify
for cnt in contours:
    epsilon = 0.02 * cv2.arcLength(cnt, True)          #
2% of perimeter
    approx = cv2.approxPolyDP(cnt, epsilon, True)

    if len(approx) == 3:
        shape_name = "Triangle"
    elif len(approx) == 4:
        shape_name = "Rectangle"

    cv2.drawContours(image, [approx], 0, (0, 255, 0), 2)
```

Retrieval modes worth knowing:

- `RETR_EXTERNAL` — returns only the outermost contours (e.g. the outline of a donut, ignoring the hole).
- `RETR_TREE` — returns every contour and reconstructs the full parent-child hierarchy of nested contours.

2.5 Application Blueprints — From Function to Working Application

By this point in the semester, you know enough individual OpenCV functions. What is still missing is the architecture that strings them together into something that actually *does* something. These three blueprints are the structured build sheets you will use for the Practical Assessment (40%) — each maps a required syllabus task directly to the OpenCV tools that accomplish it.

📋 *Lab Blueprint A — Object Tracking (Demonstrate)*

Objective: Amir's guardhouse needs to track a moving vehicle across the camera's field of view. He starts small: tracking a coloured object, such as a blue pen, across live video frames.

Stage	Implementation Detail	Syllabus Link
Input	`cap.read()` inside a `while` loop	2.1.2
Colour space	Convert frame to HSV: `cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)`	2.2.3
Masking	Define a range (`lower_blue`, `upper_blue`), then apply `cv2.inRange()`	2.3.1
Clean-up	Use `cv2.erode()` and `cv2.dilate()` to remove stray noise pixels	2.3
Localisation	`cv2.findContours()` on the mask, then find the centroid using image moments: Cx = M10/M00, Cy = M01/M00	2.4
Visualisation	Draw a circle at `(Cx, Cy)` using `cv2.circle()`	2.1

This blueprint is the logical architecture behind your code — it is the bridge between individual functions and a complete application. Every project in Unit 4 follows this same "stage-by-stage" thinking.

📋 *Lab Blueprint B — Optical Character Recognition, or Reading a Number Plate (Employ)*

Objective: Employ Tesseract OCR to read text from an image — the exact task Amir's guardhouse needs for automatic number-plate logging.

Stage	Implementation Detail	Critical Success Factor
Engine	`import pytesseract`	Ensure the Tesseract executable

Stage	Implementation Detail	Critical Success Factor
		is on the system PATH
Preprocessing	Convert to grayscale → apply **Otsu's thresholding**	Tesseract fails badly on noisy backgrounds; strict binarisation is essential
Execution	`text = pytesseract.image_to_string(thresh_img)`	Pass the pre-processed *binary* image, never the raw colour image
Output	Print or log the text	Verify the output against a known ground truth plate number

📋 *Lab Blueprint C — QR Code Recognition (Construct)*

Objective: Construct an application that decodes a QR code and takes an action — for example, a visitor QR pass that opens the guardhouse barrier.

Component	Code Snippet
Initialise	`detector = cv2.QRCodeDetector()`
Detect	`data, bbox, _ = detector.detectAndDecode(image)`
Validate	`if bbox is not None:` — always confirm a code was actually found before acting
Visual feedback	Use `cv2.line()` to draw the bounding box `bbox` around the detected code
Action	`print(f"Decoded: {data}")` — or trigger whatever real action the payload should cause

> 🎯 **Assessment Link — Unit 2**
>
> This unit prepares you for: - **Practical Work (40%):** implementing the Object Tracking or QR Code blueprint end to end. - **Mini Project (Practical Skill, 10%):** building an input loop that handles video errors gracefully instead of crashing. - **Self-Learning Task (CLO3):** research `cv2.adaptiveThreshold()`. Write a paragraph explaining why it outperforms simple thresholding on images with uneven lighting — such as a scanned document with a shadow across one corner.

UNIT 3 — Machine Learning in Computer Vision: Giving the Machine Its Judgement

Amir's Notebook, Week 8

Amir's shape detector worked perfectly on the guardhouse's plain white barrier arm. It failed completely the moment his lecturer held up a photo of a stop sign printed on a crumpled poster. No amount of tweaking the `epsilon` value in `approxPolyDP` fixed it — the rules Amir had hand-coded simply didn't generalise to a shape that wasn't perfectly clean.

"You've been telling the computer the rules," his lecturer said. *"From here, you're going to show it examples, and let it find the rules itself."*

That sentence is the entire subject of Unit 3.

> 📎 **Practical Work Link** — This unit pairs with **Practical Work 4 (Model Training Using Teachable Machine & TensorFlow)** in *Amir's Lab Companion*.

3.1 Terminology and the Machine Learning Hierarchy

Three terms are often used loosely and interchangeably in casual conversation. In this course, they are precise, nested categories:

1. **Artificial Intelligence (AI)** — the broad discipline of building machines capable of intelligent behaviour.
2. **Machine Learning (ML)** — a subset of AI in which systems learn patterns from data rather than following explicit, hand-written rules. Instead of programming "if the pixel is red, it's an apple," you show the system thousands of apple images and let it discover the characteristics itself.
3. **Deep Learning (DL)** — a subset of ML built on Artificial Neural Networks (ANNs) with many hidden layers. DL excels on unstructured data like images and video, because it *automatically* discovers useful features (edges, textures, shapes) that traditional ML required a human engineer to identify by hand.

3.2 Learning Methods: The Four Pillars

- **Supervised Learning** — the model trains on labelled data (input plus the correct answer). This covers **Classification** (predicting a category — dog vs. cat) and **Regression** (predicting a continuous value — the coordinates of a bounding box).
- **Unsupervised Learning** — the model finds structure in *unlabelled* data, most commonly through **Clustering**, grouping similar items together without ever being told what the groups mean.

- **Semi-Supervised Learning** — a blend: a small amount of labelled data combined with a large pool of unlabelled data. Think of a teacher demonstrating a handful of solved problems (labelled), after which the student practises independently on thousands of unsolved ones (unlabelled) to master the pattern. This is especially valuable in medical imaging, where expert-labelled scans are expensive but raw scans are abundant.
- **Reinforcement Learning** — an agent learns through interaction with an environment, earning rewards for good actions and penalties for bad ones. This is the paradigm behind robot navigation and game-playing agents.

3.3 Algorithms and the Objective Functions That Train Them

3.3.1 Classic Algorithms, Visualised

- **Support Vector Machines (SVM)** — find the widest possible boundary (a *hyperplane*) separating two classes. Picture drawing a road between two villages: the SVM tries to make the road as wide as possible without touching any houses on either side. The road's width is the **margin**; the houses that just touch its edge are the **support vectors**.
- **Decision Trees** — a flowchart. Start at a **root node** ("Is the pixel red?"), branch on yes/no answers, and arrive at a **leaf node** holding the final classification ("Apple").
- **Linear Regression** — predicts a continuous value by fitting the best straight line through the data, minimising the total distance between the line and every data point.

3.3.2 Loss Functions: Measuring "How Wrong"

- **Mean Squared Error (MSE)** — used in regression. Squares the difference between predicted and actual values, which heavily penalises large outliers.
- **Cross-Entropy Loss** — used in classification. Measures the distance between two probability distributions: the model's predicted class probabilities versus the true class.

3.3.3 Training Styles

- **Batch Training** — the model updates its weights only after processing the *entire* dataset. Accurate, but computationally heavy.
- **Mini-Batch Training** — the model updates weights after small groups of data (e.g., 32 images at a time). This is the standard for deep learning, balancing training stability against speed.
- **Online Training** — the model updates after seeing *each* individual data instance, useful for streaming video where lighting conditions drift gradually and the model must keep adapting.

3.4 The Machine Learning Workflow

Building an ML application follows a disciplined, repeatable pipeline:

1. **Data Collection** — gather a representative image dataset.
2. **Labelling / Annotation** — mark the ground truth, e.g. drawing boxes around every vehicle in a set of guardhouse photos.

3. **Model Selection** — choose an appropriate algorithm (SVM, Decision Tree, CNN, …).
4. **Training** — the model adjusts its internal weights to minimise the gap between its predictions and the ground truth.
5. **Evaluation** — test on unseen data using **Accuracy** (overall correctness), **Precision** (avoiding false positives), and **Recall** (avoiding false negatives).

> **Go Deeper with Ansari** — Chapter 4, *Building a Machine Learning–Based Computer Vision System*, walks through the full pipeline — image processing, feature extraction (colour histograms, HOG, LBP), feature selection, model training, and deployment — in far more mathematical detail than this course requires, but invaluable if you intend to pursue this field beyond the diploma.

3.5 Frameworks and Tools You Will Actually Use

- **Teachable Machine** — a free, browser-based Google tool that trains an image classifier directly from your webcam and exports it as a TensorFlow Lite or Keras model for use in Python. No coding is required to train the model itself.
- **TensorFlow / Keras** and **PyTorch** — the two dominant deep learning frameworks. TensorFlow/Keras is the industry standard taught in this course; PyTorch is favoured heavily in research for its flexibility.
- **Google Colab** — a free, cloud-based notebook environment providing GPU access. A GPU's parallel-processing architecture handles the massive matrix multiplications of deep learning far faster than a CPU, which is why training almost always happens on Colab rather than a laptop.

Edge computing — deployment often happens on small, low-power devices ("the edge") rather than in the cloud:

- **Raspberry Pi** — low-cost and general-purpose, adequate for simple OpenCV tasks but underpowered for heavy deep learning inference.
- **NVIDIA Jetson Nano** — purpose-built for AI, with a GPU capable of running CUDA-accelerated neural networks, making it significantly faster for real-time object detection than a Raspberry Pi.
- **Google Coral** — a USB accelerator using a Tensor Processing Unit (TPU) to run TensorFlow Lite models at very high speed and very low power draw.

3.6 Practical Training Exercises

📋 *Lab Blueprint D — Training a Classifier with Teachable Machine*

Objective: Train a "Mask vs. No Mask" classifier with no code, then run it live inside a Python script.

Step-by-step:

1. **Gather.** Open https://teachablemachine.withgoogle.com and create a new *Image Project*.
2. **Record.** Create two classes — "Mask" and "No Mask." Record at least 50 images per class using your webcam. Vary your head angle and distance from the camera as you record; a classifier trained on one fixed pose will fail the moment a real person moves naturally.

3. **Train.** Click *Train Model* and watch the Epochs counter. Confirm the Loss graph trends downward — if it does not, your two classes are probably too visually similar, or you need more varied examples.
4. **Export.** Click *Export Model* → *TensorFlow (Keras)* → *Download*. You will receive a `.h5` file.
5. **Run it in Python.**

Python

```python
import cv2
import numpy as np
from keras.models import load_model

# Load the exported model
model = load_model('keras_model.h5', compile=False)

# Preprocessing must match Teachable Machine's own logic exactly
def preprocess(image):
    image = cv2.resize(image, (224, 224))
    image_array = np.asarray(image)
    # Normalise to the [-1, 1] range — NOT [0, 255]
    normalized_image = (image_array.astype(np.float32) / 127.0) - 1
    return normalized_image.reshape(1, 224, 224, 3)

img = cv2.imread("test_face.jpg")
data = preprocess(img)
prediction = model.predict(data)
print(f"Confidence Scores: {prediction}")
```

> **Common mistake:** forgetting normalisation. Teachable Machine expects pixel values scaled to **[-1, 1]**, not the raw **[0, 255]** range OpenCV loads by default. Skipping this step is the single most common reason a student's exported model "works in the browser but not in Python."

📋 *Lab Blueprint E — Training YOLOv8 on Google Colab (GPU)*

Objective: Train a custom object detector — for instance, Amir's hard-hat and safety-vest detector for the guardhouse — using cloud GPUs.

Step-by-step:

1. **Environment.** Open a new Google Colab notebook. Go to *Runtime* → *Change Runtime Type* → select **GPU** (T4).
2. **Installation.** Run `!pip install ultralytics`.
3. **Dataset structure — read this step twice.** This is the single most common source of student errors:
   - Create a folder named `datasets` in the Colab file browser.

- Inside it, create `train` and `val` folders, each containing `images` and `labels` subfolders.
- Create a `data.yaml` file with this exact structure:

YAML

```yaml
path: /content/datasets/my_project      # absolute path
to the dataset root
train: train/images
val: val/images
names:
  0: hardhat
  1: vest
```

4. **Training command:**

Python

```python
from ultralytics import YOLO

# Load a pretrained model (transfer learning — start from
a model that already
# understands general shapes, rather than training from
nothing)
model = YOLO("yolov8n.pt")

# Train
model.train(data="/content/datasets/my_project/data.yaml"
, epochs=20, imgsz=640)
```

5. **Troubleshooting.** If you see `FileNotFoundError`, verify that the path inside `data.yaml` is **absolute** (starting with `/content/...`), and that your `images` and `labels` folders are not empty.

> **Go Deeper with Ansari** — Chapter 6, *Deep Learning in Object Detection*, traces the full family tree of detection architectures this course only summarises — R-CNN, Fast R-CNN, Faster R-CNN, Mask R-CNN, SSD, and the full YOLO lineage through YOLOv7 — including a complete walkthrough of training on Google Colab with GPU and exporting to ONNX and TensorFlow Lite.

> 🎯 **Assessment Link — Unit 3**
>
> This unit prepares you for: - **Mini Project (Knowledge, 25%):** explaining the full workflow — dataset → training → evaluation — in your own words. - **Practical Work (40%):** demonstrating a working model exported from Colab or Teachable Machine. - **Self-Learning Task (CLO3):** explore the "YOLOv8 model zoo" on GitHub. Compare `yolov8n` (Nano) against `yolov8x` (Extra Large), and write three sentences on the trade-off between inference speed (FPS) and accuracy (mAP).

UNIT 4 — Applied Projects: Assembling the Guardhouse

Amir's Notebook, Week 12

By Week 12, Amir had four separate scripts scattered across his desktop — a shape detector, a face detector, a half-working YOLO notebook, and a pose-estimation demo he'd copied from a tutorial without fully understanding. His lecturer's feedback was blunt: *"You don't have a project. You have four demos."* The difference between a demo and a project, Amir learned that week, is documentation, error handling, and the discipline to combine small working parts into one coherent system. This unit walks through the four building blocks Amir eventually assembled into his final guardhouse submission.

> 📎 **Practical Work Link** — This unit pairs with **Practical Work 5 (Object, Face, and Pose Detection with YOLOv8 via VS Code and Google Colab)** in *Amir's Lab Companion*, which walks through connecting VS Code to a free cloud GPU and running the full YOLOv8 pipeline.

4.1 Project A: Shape Detection (Geometric Logic)

This project classifies shapes using contour approximation — the culmination of Unit 2's contour tutorial.

Python

```
import cv2
import numpy as np

def detect_shapes():
    # Load image and convert to grayscale
    img = cv2.imread('shapes.png')
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Thresholding to create a binary image
    _, thresh = cv2.threshold(gray, 240, 255,
cv2.THRESH_BINARY)

    # Find contours
    contours, _ = cv2.findContours(thresh, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)

    for cnt in contours:
        # Approximate the contour
        epsilon = 0.04 * cv2.arcLength(cnt, True)
        approx = cv2.approxPolyDP(cnt, epsilon, True)

        # Use the bounding box to anchor the label text
        x, y, w, h = cv2.boundingRect(approx)
```

Python

```
        # The number of vertices determines the shape
        if len(approx) == 3:
            cv2.putText(img, "Triangle", (x, y),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0))
        elif len(approx) == 4:
            aspect_ratio = float(w) / h
            label = "Square" if 0.95 <= aspect_ratio <=
1.05 else "Rectangle"
            cv2.putText(img, label, (x, y),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0))
        elif len(approx) > 10:
            cv2.putText(img, "Circle", (x, y),
cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0))

    cv2.imshow("Shapes", img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Explanation: `epsilon` controls approximation accuracy — 4% of the arc length is a reliable starting point. A polygon approximated to three vertices is, geometrically, a triangle; distinguishing a square from a rectangle then requires one more check, the aspect ratio of its bounding box.

4.2 Project B: Face Detection (Haar Cascades)

This project implements the classic Viola-Jones algorithm using OpenCV's pre-trained XML classifiers — the same mechanism Amir's guardhouse eventually used to greet returning staff.

Python

```
import cv2

def detect_faces():
    # Load the pre-trained cascade classifier
    face_cascade =
cv2.CascadeClassifier('haarcascade_frontalface_default.xm
l')

    cap = cv2.VideoCapture(0)

    while True:
        _, frame = cap.read()
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        faces = face_cascade.detectMultiScale(gray,
```

```python
scaleFactor=1.1, minNeighbors=5)

        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y + h),
(255, 0, 0), 2)

        cv2.imshow('Face Detection', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()
```

Insight: `scaleFactor` compensates for faces appearing smaller as they move further from the camera. `minNeighbors` sets how many overlapping candidate rectangles are required before a detection is kept — higher values reduce false positives, but risk missing genuine faces at difficult angles.

4.3 Project C: Custom Object Detection with YOLOv8

For robust detection of a specific, custom object — such as hard hats on a construction site, or vehicles at Amir's guardhouse gate — the YOLO (You Only Look Once) architecture is the tool of choice.

Workflow on Google Colab:

1. **Annotate.** Use a tool such as Roboflow to draw bounding boxes around the objects in your training images, and export the dataset in "YOLOv8" format.
2. **Set up Colab.** Select a GPU runtime.
3. **Install Ultralytics.** `!pip install ultralytics`
4. **Train.**

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")                          # load a
pretrained model
results = model.train(data="data.yaml", epochs=100,
imgsz=640)
```

5. **Run inference.** After training, the trained weights file (`best.pt`) is downloaded and run locally with Python for real-time inference.

4.4 Project D: Pose Estimation (MediaPipe)

Google's MediaPipe framework tracks human joint coordinates in real time, without any custom training required.

Python

```python
import cv2
import mediapipe as mp

mp_pose = mp.solutions.pose
pose = mp_pose.Pose()
mp_draw = mp.solutions.drawing_utils

cap = cv2.VideoCapture(0)

while True:
    success, img = cap.read()
    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    results = pose.process(img_rgb)

    if results.pose_landmarks:
        mp_draw.draw_landmarks(img,
results.pose_landmarks, mp_pose.POSE_CONNECTIONS)

    cv2.imshow("Pose Estimation", img)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Application: This is the technology behind "virtual gym" apps that count exercise repetitions automatically, by calculating the angle formed between the hip, knee, and ankle landmarks across successive frames.

> **Go Deeper with Ansari** — Chapters 7 and 8, *Practical Example: Object Tracking in Videos* and *Practical Example: Face Recognition*, walk through complete, production-style implementations — including serving a live video stream in a web browser with Flask, and training a full FaceNet-based recognition model on the VGGFace2 dataset — for readers who want to push these four projects toward a genuine deployable product.

UNIT 5 — Active Learning and Assessment

5.1 Active Learning Strategies

Passive reading does not build the intuition this subject requires. These three classroom activities are designed to make abstract ideas physical.

Activity 1 — The Neural Network Roleplay *(Concept: Forward Propagation)* Students stand in groups representing network layers (Input, Hidden, Output). Input students hold numbers. Hidden-layer students hold "weight" cards and calculators. Input students pass their numbers to the Hidden layer, who multiply by their weights, sum the results, and apply a threshold (an activation function) before passing the result onward to Output. Physically watching data transform through a network demystifies what a neural network is actually doing.

Activity 2 — The Classification vs. Regression Sorting Hat *(Concept: distinguishing ML problem types)* The instructor reads out scenarios — "predicting a stock price," "diagnosing a tumour," "recognising a handwritten digit" — and students physically move to one side of the room for Classification, the other for Regression. Discuss why predicting a stock price is regression (a continuous number) while recognising a digit is classification (a discrete category, 0–9).

Activity 3 — The Teachable Machine Deployment Challenge Train a model on Teachable Machine to distinguish "Pen" from "Phone," export it as Keras (`.h5`), and write a Python script that loads the model and runs it live on a webcam. The crucial step — and the one students most often skip — is implementing Teachable Machine's specific preprocessing (resize to 224×224, normalise to [-1, 1]) exactly as shown in Lab Blueprint D. Skipping it reinforces, by direct painful experience, just how important preprocessing consistency is.

5.2 Assessment Rubric — Mini-Project Example

Criteria	Excellent (4)	Proficient (3)	Basic (2)	Limited (1)
Functionality (40%)	Runs without errors; smooth real-time performance	Runs with minor lag; core feature works	Frequent crashes or fails to detect the target	Does not run
Code Quality (20%)	Modular, well-commented, uses classes/OOP	Linear script with some comments	"Spaghetti code," no comments	Unreadable
Innovation (20%)	Extends the base tutorial with new features (e.g., data logging, a	Follows the tutorial exactly	Missing features	Incomplete

Criteria	Excellent (4)	Proficient (3)	Basic (2)	Limited (1)
	UI)			
Documentation (20%)	Clear README, dependency list, and demo video	Basic instructions provided	Instructions missing	No documentation

5.3 Final Syllabus Alignment Table

Textbook Section	Syllabus Item	CLO	Assessment Type
Unit 1.1 — The Python Paradigm	1.1 Intro to Python	CLO1	Quiz / Practical
Unit 1.4 — OOP (ShapeDetector)	1.4 OOP Features	CLO1	Practical Exercise
Unit 2.1–2.3 — Basic Operations	2.1–2.3 Image Processing	CLO1	Practical Work
Unit 2.5 — Application Blueprints	2.5 Tracking, OCR, QR	CLO2	Mini Project
Unit 3.1 — ML Terminology	3.1 Terminology	CLO2	Theory Test
Unit 3.3 — ML Algorithms	3.2 Methods (SVM/DT)	CLO2	Mini Project (Report)
Unit 3.6 — Practical Training	3.2.4–3.2.5 Workflow	CLO2	Practical Work
Self-Learning Tasks (every unit)	All Independent Learning	CLO3	Continuous Assessment

UNIT 6 — Ethics in AI: The Responsibility That Comes With Sight

Amir's Notebook, Final Week

Amir's guardhouse system worked. It recognised staff faces, tracked vehicles, read plates, and flagged visitors without safety vests. But at his final presentation, his lecturer asked a question Amir hadn't prepared for: *"Whose faces did you train it on?"*

Amir's dataset — collected from his own classmates over three weeks — was almost entirely people of one skin tone, one age range, and one gender balance. His system hadn't just learned to detect faces. It had learned to detect *faces like the ones it had seen*, and it performed noticeably worse on anyone outside that narrow range. That was the moment the course's final unit stopped being abstract.

- **Bias.** If a face-detection dataset contains only light-skinned faces, the resulting model will fail to detect darker-skinned faces reliably. This is not merely a technical shortfall — it is an ethical failure, and the engineer who curated the dataset carries responsibility for it. Every diploma project in this course should be tested against a deliberately diverse sample of test subjects before it is called "working."
- **Privacy.** Cameras used for detection or tracking inevitably raise privacy concerns. Learn and apply techniques such as **face blurring** (a Gaussian blur applied to the ROI of any detected face) to anonymise footage captured in public or semi-public spaces.
- **Transparency.** Deep learning models, in particular, are often "black boxes" — even their own creators cannot fully explain a single decision. In critical applications, such as medical diagnosis or access control, explainability is not optional; a system that cannot justify its decision should not be trusted to make it alone.

Conclusion

This textbook has given you the theoretical framework and the practical toolkit a Diploma engineer needs to enter the field of computer vision: Python's data structures reframed around pixels, OpenCV's manipulation toolkit, and the workflow behind training a machine learning model. You are now prepared to tackle real problems — automated quality control on a factory line, assistive technology for the visually impaired, or, like Amir, a working guardhouse prototype built one module at a time.

The distance between writing your first `cv2.imshow()` and deploying a custom-trained YOLO model on a Jetson Nano is a genuine leap in technical capability. It marks the transition from student to practitioner — and it happens by building, breaking, and rebuilding small working pieces, exactly the way this book has walked you through it.

One actionable habit before you close this book: start a GitHub repository today, and commit every lab exercise to it as you complete it. A portfolio showing a working object tracker, a working OCR reader, and a working pose estimator is, by a wide margin, the single most valuable asset you can carry into a job interview in this field.

Appendix A — Reference Map to Shamshad Ansari's *Building Computer Vision Applications Using Artificial Neural Networks* (2nd Edition, Apress)

Use this table to find the deeper, industry-grade treatment of any topic in this book.

This Book's Unit	Ansari Chapter	What You'll Find There
Unit 1 — Environment setup	Ch. 1 — Prerequisites and Software Installation	PyCharm, virtualenv, TensorFlow and OpenCV installation across Windows/macOS/Linux
Unit 2 — Pixels, colour spaces, drawing	Ch. 2 — Core Concepts of Image and Video Processing	Pixel basics, coordinate systems, drawing lines/rectangles/circles
Unit 2 — Transformations, arithmetic, masking, blurring, thresholding	Ch. 3 — Techniques of Image Processing	Resizing, rotation, bitwise operations, all four blur types, Otsu and adaptive thresholding
Unit 3 — The ML workflow	Ch. 4 — Building a Machine Learning–Based Computer Vision System	Feature extraction (colour histograms, GLCM, HOG, LBP), feature selection, model deployment
Unit 3 — Neural networks and CNNs	Ch. 5 — Deep Learning and Artificial Neural Networks	Perceptrons, activation functions, backpropagation, CNN architecture, TensorFlow fundamentals
Unit 3/4 — YOLO and object detection	Ch. 6 — Deep Learning in Object Detection	R-CNN family, SSD, full YOLO lineage (v2–v7), training on Colab, export to ONNX/TFLite
Unit 4 — Object tracking	Ch. 7 — Practical Example:	dHash identity, Hamming

This Book's Unit	Ansari Chapter	What You'll Find There
	Object Tracking in Videos	distance matching, streaming video with Flask
Unit 4 — Face detection	Ch. 8 — Practical Example: Face Recognition	FaceNet architecture, VGGFace2 dataset, real-time recognition pipeline
Unit 6 — Real deployment context	Ch. 9 — Industrial Application: Real-Time Defect Detection	End-to-end industrial deployment, image annotation with VoTT
Beyond this course	Ch. 10 — Computer Vision Modeling on the Cloud	Distributed training on GCP, Azure, and AWS with Horovod

Appendix B — Further Reading and Works Cited

1. Python Tutorial — Tutorials Point, https://www.tutorialspoint.com/python/index.htm
2. OpenCV Tutorial: A Guide to Learn OpenCV in Python — Great Learning, https://www.mygreatlearning.com/blog/opencv-tutorial-in-python/
3. ModuleNotFoundError: No module named 'cv2' — GeeksforGeeks, https://www.geeksforgeeks.org/python/modulenotfounderror-no-module-named-cv2-in-python/
4. Programming for Everybody (Getting Started with Python) — Coursera, https://www.coursera.org/learn/python
5. Real-Time Object Color Detection Using OpenCV — GeeksforGeeks, https://www.geeksforgeeks.org/python/real-time-object-color-detection-using-opencv/
6. Precedence of Operators — Runestone Academy, https://runestone.academy/ns/books/published/fopp/Conditionals/PrecedenceofOperators.html
7. When to Use a Dictionary, List, or Set — Stack Overflow, https://stackoverflow.com/questions/3489071/
8. List, Tuple, Set, and Dictionary Differences — GeeksforGeeks, https://www.geeksforgeeks.org/python/differences-and-applications-of-list-tuple-set-and-dictionary-in-python/
9. Object-Oriented Programming in Python — DataCamp, https://www.datacamp.com/tutorial/python-oop-tutorial
10. Encapsulation in Python — DataCamp, https://www.datacamp.com/tutorial/encapsulation-in-python-object-oriented-programming

11. Polymorphism and Inheritance in Python — AlmaBetter, https://www.almabetter.com/bytes/tutorials/python/python-inheritance-and-polymorphism
12. A Guide to Python's Magic Methods — rafekettler.com, https://rszalski.github.io/magicmethods/
13. OpenCV Shape Detection — PyImageSearch, https://pyimagesearch.com/2016/02/08/opencv-shape-detection/
14. HSV vs RGB for Image Processing — dev.to, https://dev.to/jarvissan22/cv2-hsv-vs-rgb-understanding-and-leveraging-hsv-for-image-processing-49ac
15. Smoothing Images — OpenCV Documentation, https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html
16. Edge Detection Using OpenCV, https://opencv.org/blog/edge-detection-using-opencv/
17. Contour Detection Using OpenCV — LearnOpenCV, https://learnopencv.com/contour-detection-using-opencv-python-c/
18. A Guide to OCR With Tesseract — Unstract, https://unstract.com/blog/guide-to-optical-character-recognition-with-tesseract-ocr/
19. QR Code Scanner Using OpenCV 4 — LearnOpenCV, https://learnopencv.com/opencv-qr-code-scanner-c-and-python/
20. Difference Between Supervised, Unsupervised, and Reinforcement Learning — NVIDIA Blog, https://blogs.nvidia.com/blog/supervised-unsupervised-learning/
21. Loss Functions in Machine Learning Explained — DataCamp, https://www.datacamp.com/tutorial/loss-function-in-machine-learning
22. Machine Learning Model with Teachable Machine — GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/machine-learning-model-with-teachable-machine/
23. Training Custom Datasets with YOLOv8 in Google Colab — Ultralytics, https://www.ultralytics.com/blog/training-custom-datasets-with-ultralytics-yolov8-in-google-colab
24. Jetson Nano vs Raspberry Pi for AI — Think Robotics, https://thinkrobotics.com/blogs/learn/jetson-nano-vs-raspberry-pi-ai-the-ultimate-performance-comparison-for-edge-computing
25. How to Train YOLOv8 on a Custom Dataset — Roboflow Blog, https://blog.roboflow.com/how-to-train-yolov8-on-a-custom-dataset/
26. Pose Detection with MediaPipe — GitHub Gist, https://gist.github.com/rmeziatisab/20820a7c8cc667a1da44f22bcbcb7923
27. AI Ethics: What It Is and Why It Matters — Coursera, https://www.coursera.org/articles/ai-ethics
28. Ansari, S. (2023). *Building Computer Vision Applications Using Artificial Neural Networks: With Examples in OpenCV and TensorFlow with Python* (2nd ed.). Apress. ISBN 978-1-4842-9865-7.

Appendix C — Map to *Amir's Lab Companion* (Practical Work 1–5)

This textbook has a companion volume, ***Amir's Lab Companion***, which contains the full step-by-step Practical Work instructions (PW1–PW5) that your Continuous Assessment is graded against — objectives, materials, exact code, troubleshooting notes, and the Problem-Based Learning challenges. Use this table to find the lab that matches whatever you're currently reading here.

This Book's Unit	Companion Practical Work	What You'll Build
Unit 1 — Foundations of Python Programming	**PW1** — Introduction to Python Programming	Python/VS Code environment, dynamic typing and control-flow exercises, the `FaceDetector`/`DetectionResult` OOP patterns, and four original Problem-Based Learning programs
Unit 1.4 (OOP) and Unit 2.1–2.2	**PW2** — Python Basics for Image Processing	Loading, displaying, saving, resizing images with NumPy and OpenCV; the reusable `ImageProcessor` OOP class
Unit 2 in full	**PW3** — OpenCV Image and Video Processing	Colour-space conversions, masking, all four blur types, Canny edge detection with a tuning graph, contour drawing and approximation, HSV object tracking, an MNIST-based OCR pipeline, and QR/barcode detection
Unit 3 (Terminology, Workflow, Tools)	**PW4** — Model Training with Teachable Machine & TensorFlow	A no-code image classifier trained and exported from Teachable Machine, real-time webcam inference, pose estimation, and a touchless MediaPipe hand-tracking volume controller
Unit 3.5–3.6 and Unit 4 (Applied Projects)	**PW5** — Object, Face, and Pose Detection with YOLOv8	Connecting VS Code to a free Google Colab GPU via Remote Tunnels, running YOLOv8

This Book's Unit	Companion Practical Work	What You'll Build
		analytics scripts, building a bounding-box privacy blur filter, and pose estimation on video

End of textbook.